

Hush Project Structure

Objective

Hush is an NYC Subway routing app that uses a Graph Neural Network to predict station congestion/noise levels and recommend quieter routes

Project Structure

Code block

```
1  hush/
2  |
3  |— requirements.txt      # Python dependencies
4  |— start_app.sh         # Shell script to launch the Streamlit app
5  |
6  |— data/
7  |   |— raw/             # Raw MTA data
8  |   |   |— 2020.csv      # MTA ridership/turnstile data for 2020
9  |   |   |— 2021.csv      # MTA ridership/turnstile data for 2021
10 |   |   |— 2022.csv      # MTA ridership/turnstile data for 2022
11 |   |   |— 2023.csv      # MTA ridership/turnstile data for 2023
12 |   |   |— 2024.csv      # MTA ridership/turnstile data for 2024
13 |   |   |— stops.csv     # MTA stop metadata (IDs, names, locations)
14 |   |   |— stop_times.csv # Scheduled stop times for all routes
15 |   |
16 |   |— processed/        # Output of preprocessing scripts
17 |   |   |— ComplexNodes.csv # Mapping from Station Complexes -> Graph
18 |   |
19 |   |— Nodes
20 |   |   |— ComplexEdges.csv # Connections between station complexes ->
21 |   |   |
22 |   |   |— Graph Edges
23 |   |   |   |— StopComplex.csv # Mapping of stops to their complex
24 |   |   |   |— StopCoords.csv  # Coordinates for each stop
25 |   |   |   |— DedupedStopCoords.csv # Deduplicated stop coordinates
26 |   |   |   |— stats.csv        # Normalization stats (mean/std) for model
27 |   |   |
28 |   |   |— input
29 |   |   |   |— eval.csv         # Evaluation metrics from model assessment
30 |   |   |   |— train.parquet    # Training dataset
31 |   |   |   |— val.parquet      # Validation dataset
32 |   |   |   |— test.parquet     # Test dataset
33 |   |
34 |   |— models/            # Trained GNN models with different
35 |   |   |— hyperparameters
```

```

29 | | | best_model.pt # Best model based on WMAPE
30 | | | best_model_config.json # Model config
31 | | | model_1.pt
32 | | | ...
33 | | | model_8.pt
34 |
35 | | training/ # Offline GNN training
36 | | | preprocess.py # Master preprocessing pipeline script
37 | | | train.ipynb # Define, train and save GNN
38 | | | evaluate.ipynb # Evaluate model performance
39 | | | preprocessing/ # Individual preprocessing step scripts
40 | | | | build_gtfs_sequences.py # Build routes
41 | | | | build_complex_edges.py # Builds graph edges between complexes
42 | | | | build_stop_complex.py # Creates stop-to-complex mapping
43 | | | | build_stop_coords.py # Extracts and saves stop coordinates
44 | | | | deduplicate_stop_coords.py # Removes duplicate stop coordinates
45 |
46 | | | src/ # Hush application
47 | | | | __init__.py # Marks src as a Python package
48 | | | | app.py # Streamlit entry point
49 | | | | congestion_scorer.py # Scores routes based on predicted
congestion
50 | | | | gnn_inference.py # Loads model and runs inference to get
quiet scores
51 | | | | linear_css.py # Injects custom CSS into Streamlit
52 |
53 | | | | AppHelpers/ # Helper modules
54 | | | | | gnn_loader.py # Loads the trained GNN model and graph data
55 | | | | | routing.py # Pathfinding logic across the station graph
56 | | | | | station_data.py # Loads and exposes station/coordinate data
57 |
58 | | | | Components/ # Streamlit UI components
59 | | | | | Header.py # App header
60 | | | | | MapComponent.py # Interactive map showing stations and
routes
61 | | | | | SearchInputs.py # Origin/destination search input fields
62 | | | | | SearchResults.py # Displays ranked route results with quiet
scores
63 | | | | RouteCard.py # Individual route result card

```

Stage 1: Offline Training

Code block

```

1 Raw MTA Data (2020-2024.csv, stops.csv, stop_times.csv)
2 |

```

```

3         v
4     training/preprocess.py # Orchestrates training pipeline
5         |
6         ├── build_stop_coords.py      -> StopCoords.csv
7         ├── deduplicate_stop_coords.py -> DedupedStopCoords.csv
8         ├── build_stop_complex.py     -> StopComplex.csv
9         ├── assign_complex_nodes.py   -> ComplexNodes.csv
10        └── build_complex_edges.py     -> ComplexEdges.csv
11            |
12            v
13        data/processed/(train.parquet, val.parquet, test.parquet, stats.csv)
14            |
15            v
16        training/train.ipynb
17        # Defines GNN architecture
18        # Trains on station graph
19        # Each node has ridership features (how busy a station is over time)
20        # Model learns congestion patterns to predict a quiet score for each
station
21        # Saves trained weights
22            |
23            v
24        models/model_x.pt
25            |
26            v
27        training/evaluate.ipynb
28        # Loads test.parquet
29        # Runs model on unseen data
30        # Outputs eval.csv with performance metrics

```

Stage 2: Streamlit Application

Code block

```

1     User opens the app (start_app.sh -> streamlit run src/app.py)
2         |
3         v
4     app.py (main entry point)
5         |
6         ├── inject_custom_css()      # linear_css.py applies styling
7         ├── load_station_coordinates() # station_data.py loads
DedupedStopCoords.csv
8         ├── get_station_list()       # station_data.py loads
ComplexNodes.csv
9         |
10        └── LEFT COLUMN (UI Panel)

```

```

11      |      |—— Header()           # Header
12      |      |—— SearchInputs()      # User selects origin and
      destination station
13      |      |—— SearchResults()      # Displays routes with quiet scores
14      |      |      |
15      |      |      v
16      |      |      get_routes()      # routing.py runs pathfinding on the
      graph
17      |      |      |
18      |      |      v
19      |      |      gnn_loader.py      # Loads best model
20      |      |      |
21      |      |      v
22      |      |      gnn_inference.py   # Runs GNN to get quiet scores per
      station
23      |      |      |
24      |      |      v
25      |      |      congestion_scorer.py # Aggregates station scores into
      route scores
26      |      |      |
27      |      |      v
28      |      |      RouteCard()        # Renders each route with its quiet
      score
29      |      |
30      |      |—— RIGHT COLUMN (Map)
31      |      |      |—— MapComponent() # Shows interactive map with
      origin/destination
32      |      |

```

```

# Highlights recommended route

```

Model Choice

- **Graph Neural Network** with a **custom DirSAGEEmbRes architecture**
 - Built on top of GraphSAGE layers
 - Direction-aware message passing, station-specific identity, stable deep propagation
 - Dir = Directed graph, separating incoming and outgoing message passing
 - SAGE = Based on GraphSAGE
 - Emb = Includes learnable node embeddings
 - Res = Residual connection
- Ridership at a station depends on
 - It's own history + time-of-day/day-of-week -> temporal
 - Nearby stations -> spatial spillover

- Directional effects (inflow and outflow of commuters)

Learnable Node Embeddings	• Gives the model a trainable station identity vector • Represent things like ◦ Baseline demand (quiet vs busy stop) ◦ Being a transfer hub ◦ Neighbourhood differences
Two SAGEConv Layers	1. Two layers -> incorporate information from neighbours up to 2 hops away • After 1st layer ◦ Each node updates representation using current features and features from its direct neighbours -> node's embedding contains 1-hop information • After 2nd layer ◦ Node again aggregates from its direct neighbours ◦ Neighbour's embeddings already contain information from their neighbours from the first layer -> receive information from 2-hop neighbours 2. Separate convolutions for incoming and outgoing edges • Model learns different weights for each direction • A station may have ◦ Incoming > outgoing in the morning (industrial area) ◦ Incoming > outgoing at night (residential area) • Better modelling of asymmetric flow patterns
Residual Connections	• Instead of a layer producing the next layer entirely from scratch, produce an update to the existing representation • Constantly averaging with neighbours can cause nodes in the same region to start to look more alike -> over-smoothing -> node embeddings become too similar after many GNN layers -> model can no longer distinguish nodes well -> gradient fading, harder optimisation • Stations connected through short paths -> especially high risk of over-smoothing • Residuals preserve the node's original representation (ridership and embedding) -> layers learn corrections instead of rewriting -> stack layers without removing station identity
ReLU Activation	Activation • Without activation -> Stack layers that are only linear transformations -> collapses to a single layer transformation • Relationships are unlikely to be linear

- Activation after first SAGEConv -> feed a non-linear representation into the next layer -> second layer build complex patterns

ReLU

- Hidden dimensions can represent patterns like
 - Strong incoming congestion
 - Station behaving unusually for this hour
 - High transfer-related activity
- Passes forward useful detected patterns
- Zeroes out unhelpful opposite-direction patterns
- Changes computation from a global linear rule to a composition of piecewise linear rules

1. Concatenate node features with learnable node embedding
 - a. Each station gets raw features merged with its trainable identity vector
2. Apply first incoming/outgoing SAGEConv -> RELU
 - a. Each node aggregates features from stations flowing into it, then stations it flows into
3. Apply second incoming/outgoing SAGEConv -> ReLU -> Residual
 - a. Each node incorporates information from 2-hop neighbours
4. Concatenate the two directional outputs
 - a. Concatenate representations, not add
 - b. Adding would average them and lose the directional distinction -> concatenation preserves them independently -> final layer decides how much weight to give each direction per station
5. Pass through the final linear layer
 - a. Learns a weighted combination of the features for each node
 - b. Outputs the predicted normalised ridership for time $t+1$ (scalar)

Model Training

Graph Snapshots

- Train network structure is fixed -> fixed nodes and edges
- For each pair of consecutive hours in the data, create a pair (input features at t , target ridership at $t+1$)

- Some timestamps might be missing certain stations -> Allocate dense tensors for X and y, represent a missing station row as all zeros

Adam Optimiser

- Adaptive learning rate optimiser, uses backpropagation gradients for each parameter to decide the direction to move parameters and the learning rate
- Keeps track of two running statistics for each parameter's gradient
- Moving average of the gradient
 - General direction the parameter has been wanting to move recently
 - Smooths over recent gradients instead of reacting only to the current one
- Moving average of the squared gradient
 - How large or volatile the gradients tend to be for that parameter
 - Consistently large -> reduce step size
 - Consistently small -> increase step size
- Different parameters may behave differently -> use Adam to allow each parameter to have its own learning rate

Training Step For One Snapshot

1. Clear old gradients stored from the previous training step
 - a. If not, new gradients would be added onto old ones by mistake
2. Forward pass, runs the snapshot through the model, output predicted ridership
3. Compute loss
4. Backpropagation, compute gradients of loss wrt every trainable parameter in the model
 - a. How each SAGEConv weight, node embedding, linear layer weight affected the loss
5. Optimiser step, Adam uses gradients to update parameters
6. Add snapshot loss to epoch total
7. Training loss -> how well training went in the epoch

Validation Step For One Snapshot

1. Run model on validation snapshots
2. Predict on validation snapshots
3. Compute and accumulate validation loss
4. Save the model if validation improved

Early Stopping For Model Training

- Compares the new validation loss to the best one seen so far
- Stops early if validation does not improve for PATIENCE epochs in a row

Metadata Saving

- After training a model, save run metadata to a JSON file
- Keep track of which configuration produced which model

Model Hyperparameter Tuning

- Train a model for each parameter configuration, save results and metadata
- Compare different hyperparameter settings and pick the best performing model
- Configurations used

Learning Rate	Hidden dimensions	Embedded dimensions	Loss Function
1e-4	64	16	MSE
1e-4	64	16	Huber
3e-4	64	16	MSE
3e-4	64	16	Huber
1e-4	128	32	MSE
1e-4	128	32	Huber
3e-4	128	32	MSE
3e-4	128	32	Huber

- Learning Rate -> more conservative/aggressive update speed
- Hidden dimension -> model capacity, smaller/larger network
- Embedding dimension -> richness of station identity
- Loss function -> see which is more robust for the regression task

Model Evaluation Metrics

1. Turn test data into graph snapshots

2. For each model, rebuild the graph with the saved dimensions
3. Compute the prediction per snapshot
4. Compute metrics
5. Rank all models by WMAPE > MAE > RMSE
6. Save ranked CSV to models, pick the best model
7. Re-evaluate the best model to produce detailed results

Overall Metrics	Error by hour	Per-station breakdown
MSE/MAE normalised	For each hour	• MAE
MSE/MAE/RMSE raw	• MAE	• Average ridership
MedAW	• Count	• MAPE
R^2	• Bar chart	• n predictions • Top 10 worst • Top 10 best • MAPE distribution

Design Decisions

Mask	• Only stations actually present at t+1 are scored • Avoid penalising the model for missing data zeros
Denormalisation	• Predictions are converted back to real tap-in counts using per-station mean and std from training stats
Baseline comparison	• Model is always benchmarked against simply predicting each station's historical mean • Informs us if the model adds value

Model Evaluation Results

- Models are ranked based on WMAPE, then MAE, then RMSE
- Further details on the best model are then displayed
- Best model was `model_6 (lr=0.0001, hidden_dim=128, emb_dim=32, loss=huber)`

mse_norm	mae_norm	r^2	mae	med_ae

0.1178	0.1970	0.9652	47.22	17.37
--------	--------	--------	-------	-------

Analysis of Results

Overall Metrics	<pre>===== OVERALL METRICS ===== Normalized space: MSE = 0.1178 MAE = 0.1970 Raw space: MSE = 17273.69 RMSE = 131.43 MAE = 47.22 tap-ins MedAE = 17.37 tap-ins R^2 = 0.9652 === Baseline: station mean === MAE = 235.27 WMAPE = 71.18% === Improvement vs baseline (station mean) === delta_MAE = 188.04 delta_WMAPE = 56.89%</pre>
Huber loss >> MSE loss	- Occasional large spikes - peak hour traffic, unusually busy stations, occasional sudden jumps - MSE - Heavily emphasises large mistakes - These few large mistakes dominate the gradient and steer training too strongly towards them - Model spends too much effort training on extremes, sacrificing doing well on most ordinary cases - Huber - Squared error for small errors, absolute error for large errors - Large outliers still matter, but do not completely dominate training - Much less dominated by spikes/peaks than MSE

- Generally accurate most of the time, not destroyed by occasional unusual spikes
- Huber performing better than MSE -> dataset probably has outlier-like cases
- Occasional large spikes - peak hour traffic, unusually busy stations, occasional sudden jumps
- MSE
 - Heavily emphasises large mistakes
 - These few large mistakes dominate the gradient and steer training too strongly towards them
 - Model spends too much effort training on extremes, sacrificing doing well on most ordinary cases
- Huber
 - Squared error for small errors, absolute error for large errors
 - Large outliers still matter, but do not completely dominate training
 - Much less dominated by spikes/peaks than MSE
 - Generally accurate most of the time, not destroyed by occasional unusual spikes
- Huber performing better than MSE -> dataset probably has outlier-like cases

Hour	MAE	Count	Bar
00:00	18.29	149280	
01:00	17.89	146720	
02:00	23.29	145604	
03:00	22.12	147719	
04:00	18.59	151917	
05:00	26.83	152980	
06:00	44.25	153202	
07:00	86.04	153251	
08:00	113.60	153164	
09:00	49.72	153139	
10:00	45.44	153117	
11:00	42.01	153082	
12:00	40.72	153107	
13:00	43.07	153168	
14:00	50.84	153247	
15:00	65.47	153254	
16:00	70.10	153210	
17:00	104.79	153154	
18:00	56.03	153140	
19:00	42.63	153044	
20:00	36.57	152967	
21:00	38.06	152888	
22:00	40.69	152579	
23:00	31.79	151862	

10 best and 10 worst stations

Top 10 worst stations (highest MAPE):					
Station	MAE	Avg Ridership	MAPE%	n	
448	139.54	230.37	60.6%	8774	
202	4.97	10.23	48.6%	7188	
83	34.27	71.43	48.0%	7979	
85	26.55	58.73	45.2%	7727	
453	117.16	270.57	43.3%	8781	
199	3.14	8.21	38.2%	7717	
201	4.76	13.01	36.6%	7917	
86	12.97	36.63	35.4%	7556	
200	7.31	22.36	32.7%	8295	
604	207.92	691.40	30.1%	8781	
Top 10 best stations (lowest MAPE):					
Station	MAE	Avg Ridership	MAPE%	n	
614	204.08	1935.46	10.5%	8781	
630	63.54	606.13	10.5%	8781	
263	36.32	347.71	10.4%	8780	
68	20.04	194.88	10.3%	8770	
623	129.18	1258.65	10.3%	8781	
602	264.44	2613.00	10.1%	8781	
259	55.41	575.90	9.6%	8781	
447	161.46	1684.88	9.6%	8755	
51	45.52	487.00	9.3%	8781	
451	68.72	743.93	9.2%	8781	

Application Route Finding & Scoring

User Input	<code>app.py</code> -> <code>SearchInputs.py</code>	- User selects origin and destination station names from a dropdown - Station names are mapped to station IDs via <code>get_station_list()</code> which reads <code>StopCoords.csv</code>
Coordinate Lookup	<code>routing.py</code> -> <code>station_data.py</code>	<code>origin_name</code> -> <code>origin_id</code> (e.g. "101N") -> <code>get_station_coords()</code> -> { lat, long } - Tries exact ID match first -> fall back to stripping N/S suffix -> try N suffix
Google Routes API Call	<code>routing.py</code>	- Subway/rail only - Requests alternative routes - Returns raw route data including steps, durations, transit line info
Route Processing	<code>routing.py</code>	For each raw route - Extract transit steps - line name, colour, departure/arrival station, stop count, duration - Extract walk steps - distance, duration - Merge consecutive walk steps - Filter out walks under 1 minute - Deduplicate routes by signature (line, departure, arrival) tuples - Sort by duration, take top 3 <code>top_routes</code> - Get intermediate stops for each root
Intermediate Steps	<code>intermediate_lookup.py</code>	- Returns intermediate stop names between departure and arrival on a given route - Lookup all known stop sequences for the <code>route_id</code> - For each sequence, match departure and arrival names, slice sequence between them - Return the longest valid intermediate list found across all sequences - Return [] if nothing matched
GNN Congestion	<code>gnn_loader.py</code> -> <code>gnn_inference.py</code>	<code>top_routes</code> <code>calculate_route_quiet_scores(top_routes)</code>

Prediction		a. <code>predictions = get_tap_in_predictions()</code> : Dict mapping <code>complex_id</code> -> <code>predicted_tap_ins</code> i. Filter rows matching current hour + day of week from 2024.csv, randomly sample 1 row per complex ii. <code>load_gnn_predictor()</code> -> <code>get_predictor()</code> -> <code>GNNPredictor()</code> , initialise predictor with nest model and mappings iii. <code>predictor.predict()</code> , run GNN inference 3. <code>GNNPredictor.predict()</code> a. Normalise ridership per station using <code>stats.csv</code> (mean, std) b. Compute time features - sin/cos of hour/day of week c. Build feature matrix [num_nodes, 5] i. <code>ridership_norm</code>, <code>sin_hour</code>, <code>cos_hour</code>, <code>sin_dow</code>, <code>cos_dow</code> d. Forward pass - <code>DirSAGEEmbRes(X, edge_in, edge_out)</code> e. Denormalise - <code>pred_raw = pred_norm * std + mean</code> i. Clamp to ≥ 0 4. Predictions - { <code>station_complex_id</code> -> <code>predicted_tap_ins_next_hour</code> } 5. Results cached for 5 minutes via <code>@st.cache_data(ttl=300)</code>
Quiet Score Calculation	<code>congestion_scorer.py</code>	For each transit step in each route • For each stop -> <code>_find_station_id()</code> ◦ Direct match in StopComplex.csv -> Case-insensitive match -> Substring match -> First-word match • For each station ID <code>get_station_congestion_score()</code> ◦ <code>tap_ins = predictions[station_id]</code> ◦ <code>percentile = percentileofscore(all_tap_ins, tap_ins)</code> ◦ <code>congestion_score = percentile / 100</code> <- 0.0 (empty) to 1.0 (packed) • <code>route_congestion = mean(congestion_scores)</code> • <code>quiet_score = round((1.0 - route_congestion) * 10)</code> • <code>quiet_score = clamp(0, 10)</code>

		Congestion is relative - a station is scored against all other stations at that moment, not against an absolute threshold
Result Display	<code>SearchResults.py</code> -> <code>RouteCard.py</code>	• Routes sorted by duration • Best route = highest <code>quiet_score</code> • Each route card shows ◦ Duration, distance, transfers ◦ Per-step - line badge, departure -> arrival, stops, duration ◦ Quiet score badge ■ ≥ 7 -> Quiet ■ ≥ 4 -> Moderate ■ < 4 -> Busy

Data Preprocessing

Raw Data

year.csv	transit_timestamp, transit_mode, station_complex_id, ridership, transfers
stop_times.csv	trip_id, stop_id, arrival_time, departure_time, stop_sequence
stops.csv	stop_id, stop_name, stop_lat, stop_lon, location_type, parent_station

Processed Data

ComplexEdges.csv	from_complex_id, to_complex_id	Mapping complex connections
ComplexNodes.csv	complex_id, node_id	Mapping complexes to graph nodes
DedupedStopCoordinates.csv	name, lat, lng	Mapping stations to their coordinates (ignoring N/S)

StopCoords.csv	stop_id, lat, lng, name	Mapping stations to their coordinates
stats.csv	station_complex_id, mean, std	For normalising and denormalising raw tap-in counts for use by the model
StopComplex.csv	Stop Name, Complex ID	Map stops to their complexes
test/train/val.parquet	-	Train/Val/Test split

One-time Setup Scripts (training/preprocessing)

- build_complex_edges, build_stop_complex, build_stop_coords, deduplicate_stop_coords, build_gtfs_sequence.py

Preprocessing Pipeline (training/preprocess.py)

1. Load, clean and aggregate yearly CSVs (2020.csv - 2024.csv)
2. Apply train/val/test split
3. Compute normalisation stats
4. Build station_complex_id -> node_id mapping from train data
5. Add features and save splits
 - a. Filter to stations in ComplexNodes
 - b. Normalised ridership, time features, node_id
 - c. Hour and day of week are cyclical -> sin/cos encoding to help model see that it wraps around

Assumptions

Tap-ins are a proxy for congestion We only have tap-in data, as the New York Subway does not have tapping out	• Model learns patterns associated with future demand pressure or likely busyness, which may correlate with congestion • High tap-ins -> More people entering the system • Surge in tap-ins -> Indicates the start of a downstream load • Neighbouring station tap-ins -> Reveal flow patterns • Associate certain tap-in configurations with later busy conditions • Does not account for exits, train occupancy, delays, passenger accumulation
---	--

Network topology is fixed	• Station connectivity is stable • Service disruptions, closures, etc. are not accounted for
The next hour can be predicted from the current snapshot	• Information needed for next-hour prediction is mostly present in the current state • The prediction horizon of one hour is appropriate
A matching day and hour serves as a reasonable proxy	• Lack of live data • A matching day and hour from the most recent year (2024) is our closest approximation

Challenges

Insufficient Data	• MTA data lacks tap-out or live data, limiting our ability to provide real-time predictions or predict congestion directly • TfL lacked historical data but provided live data • Many transport APIs lacked sufficient data • Chose to use MTA as it provided us sufficient data to actually train a model
Wrong Initial Model Choice	• Initially chose an undirected graph • Model learnt that two stations influence each other, but could not distinguish ◦ Influence coming from before a station in a route ◦ Influence coming from after a station in a route • Why does direction matter? ◦ At certain times, residential-area and business-district stations may have different patterns ◦ A model may know a station is connected to busy neighbours, but not whether those signals are incoming or outgoing ◦ Poor ability to model movement patterns
Wrong Initial Loss Choice	• Started with MSE • Model performed badly at 0700, 0800,, peak hours • MSE did not handle spikes well -> use Huber loss instead

Future Improvements

- Poor performance on peak hour
 - Add better peak-specific features e.g. `morning_peak_flag`, `evening_peak_flag`, `is_holiday`
 - Reweight the loss so peak periods matter more - give more weight to peak-hour samples
- Add stronger station identity information e.g. borough/zone
- Improved temporal encoding
 - Week of year
 - Season
 - Time since service start/end